

ДИСЦИПЛИНА	Программирование (полное наименование дисциплины без сокращений)
ИНСТИТУТ	Институт тонких химических технологий им. М.В. Ломоносова
КАФЕДРА	Кафедра информационных систем в химической технологии полное наименование кафедры)
ВИД УЧЕБНОГО МАТЕРИАЛА	Практические занятия (в соответствии с пп.1-11)
ПРЕПОДАВАТЕЛЬ	Серебренников Н.П. (фамилия, имя, отчество)
СЕМЕСТР	2-й семестр, 1-й курс бакалавриата, 2024-2025 учебный год (указать семестр обучения, учебный год)

Практическая работа № 7. Создание программы для анализа химических молекул в формате SMILES.

Цель работы

Закрепить работу с функциями в Python, научиться использовать полученные ранее знания для создания прикладных приложений.

Задания

1. Постановка задачи

- Программа для анализа молекул в формате SMILES должна содержать:
 - 1) Обработку и парсинг данных из командной строки, вводимые данные в формате SMILES. Пример – «HCl», «H-O[N+](=O)[O-]», «[CO2-C-H-O3-Zn-H-C]».
 - 2) На первом шаге необходимо отобрать атомы без учёта разделителей - "[]= #/ - \ \ ()".
 - 3) Далее посчитать количество одинаковых атомов.
 - 4) Получить значение молекулярной массы соединения.
 - 5) Проанализировать молекулу по условиям:
 - а. Если в ней OH?
 - б. Есть ли двойная связь с кислородом?
 - 6) Сделать вывод о молекуле:

- a. Кислородсодержащая кислота.
- b. Жесткое основание.
- c. Соединение не определено.

Составим текстовую блок-схему для программы:

1) Функция для парсинга входных данных и записи отдельных атомов в список.

- Начало выполнения
- Создать множество разделителей
Содержит символы: []=#/-\()
- Создать пустой список atoms
Для хранения найденных атомов
- Создать пустой список atom_counts
Для хранения количества каждого атома
- Создать временную переменную temp_atom
Для формирования текущего атома из символов
- Начать цикл по каждому символу формулы с индексом i
- Проверка: символ НЕ является разделителем?
- Да → перейти к проверке символа
- Нет → пропустить, перейти к следующему символу
- Проверка: символ — заглавная буква?
 - Да →
 - Сохранить в temp_atom
 - Если следующий символ — строчная буква:
 - Добавить его к temp_atom
 - Добавить temp_atom в atoms
 - Добавить 1 в atom_counts
 - Нет → перейти к следующей проверке
- Проверка: символ — цифра?
 - Да → заменить последний элемент в atom_counts на значение этой цифры
- Переход к следующей итерации
- После завершения цикла вернуть:

- Список atoms
- Список atom_counts
- Конец выполнения

Вход: "H2O"

Выход: atoms = ['H', 'O'], atom_counts = [2, 1]

2) Функция проверки дубликатов

- Начало
- Инициализация пустого словаря unique_atoms
- Начать цикл по парам (атом, количество)
- Проверить: существует ли атом в unique_atoms?
 - Да → Увеличить значение по ключу
 - Нет → Добавить ключ с текущим количеством
- Повторять для всех пар
- Вернуть список ключей и список значений
- Конец

Вход: atoms = ['H', 'O', 'H'], atom_counts = [1, 1, 1]

Выход: ['H', 'O'], [2, 1]

3) Функция подсчета молекулярной массы

- Начало
- Попытка выполнения блока (try):
- Считать данные из CSV-файла (weight.csv)
 - Использовать np.genfromtxt
 - Разделитель: ;
- Создать словарь weight_dict
 - Ключ: название элемента (row[0])
 - Значение: атомная масса (row[2])
- Вычислить молекулярную массу:
 - Пройтись по парам (атом, количество)
 - Умножить массу элемента из weight_dict на его количество
 - Если атом не найден — масса 0
- Вернуть сумму всех произведений

- Если возникла ошибка (except):
- Вывести сообщение об ошибке
- Вернуть 0
- Конец

Пример:

Если weight.csv содержит:

H;1;1.008

O;8;16.00

и входная формула H₂O, то

молекулярная масса = $1.008 * 2 + 16.00 * 1 = 18.016$

4) Функция определения соединения

- Старт
- Проверка: содержит ли atoms элемент "O" и "H"?
 -  Нет → Перейти к п.6
 -  Да → перейти к следующей проверке
- Проверка: содержит ли formula строку "=O"?
 -  Да → вернуть: "Обнаружена кислородсодержащая кислота"
 -  Нет → перейти к п.5
- Вернуть: "Обнаружено жесткое основание"
- Вернуть: "Соединение не определено"
- Конец

Пример:

"H₂O" → "Обнаружено жесткое основание".

"H₂SO₄" → "Обнаружена кислородсодержащая кислота".

5) Функция main (точка входа).

- В Python у каждого файла есть специальная переменная `__name__`.
- Если файл запускается напрямую, то `__name__ == "__main__"`.
- Если файл импортируется в другой скрипт, то `__name__` будет равно имени файла без .py.

Пример использования

1. Запуск напрямую

bash

python script.py

 Выполнится **main()**.

2. Импорт в другой файл

import script

 **main()** не выполнится автоматически, а только если вызвать его вручную:

script.main()

Пример оформления точки входа:

!Будьте внимательны, возможно ваши переменные и функции названы иначе!

```
def main():  
    formula = input("Введите формулу: ")  
    atoms, atom_counts = parse_formula(formula)  
    atoms, atom_counts = merge_duplicate_atoms(atoms,  
atom_counts)  
    molecular_weight =  
calculate_molecular_weight(atoms, atom_counts)  
    classification = classify_compound(atoms,  
formula)  
  
    print("Атомы:", atoms)  
    print("Количество каждого атома:", atom_counts)  
    print(classification)  
    print("Молекулярная масса:", molecular_weight)  
    print("Формула:", formula)
```

```
#добавляем возможность использовать как модуль
```

```
if __name__ == "__main__":  
    main()
```

Дополнительно.

Для доступа к файлу (функция 3, см. выше) используйте блок try:

```
try:  
    elements = np.genfromtxt(weight_file,  
delimiter=";", dtype=None, encoding=None)  
    weight_dict = {row[0]: row[2] for row in  
elements}  
    ...  
    return _____  
except Exception as e:  
    print(f"Ошибка при чтении файла с весами  
элементов: {e}")  
    return 0
```

Дополнительное задание (опционально).

Создайте простой текстовый тренажёр, который помогает учить иностранные слова (например, английские слова с переводом на русский).

Программа позволяет:

- Добавлять новые слова в словарь
- Тренироваться, выбирая правильный перевод
- Сохранять словарь между сессиями в текстовом файле

Файл словаря: vocabulary.txt

Хранит слова в формате:

word:перевод

hello:привет

apple:яблоко

Основные компоненты:

main() — Главный цикл программы

Что делает:

- Отображает меню
- Ждёт выбор пользователя
- В зависимости от выбора вызывает нужную функцию

```
while True:
```

```
    print("1. Добавить слово")
```

```
    print("2. Начать тренировку")
```

```
    print("3. Выйти")
```

```
    choice = input("Выберите действие: ")
```

```
    if choice == "1":
```

```
        add_word()
```

```
    elif choice == "2":
```

```
        words = load_words()
```

```
        train(words)
```

```
    elif choice == "3":
```

```
        break
```

```
    else:
```

```
        print("Неверный ввод")
```

```
if __name__ == "__main__":
```

```
    main()
```

add_word() — Добавление нового слова

Что делает:

- Спрашивает у пользователя слово и перевод
- Сохраняет пару в файл vocabulary.txt

```
eng = input("Введите английское слово: ")
rus = input("Введите перевод: ")
with open("vocabulary.txt", "a", encoding="utf-8") as f:
    f.write(f"{eng}:{rus}\n")
```

load_words() — Загрузка слов из файла

Что делает:

- Открывает vocabulary.txt
- Читает каждую строку
- Разделяет слово:перевод и добавляет в словарь Python

```
words = {}
with open("vocabulary.txt", "r", encoding="utf-8") as f:
    for line in f:
        eng, rus = line.strip().split(":")
        words[eng] = rus
```

Возвращает словарь, ключ — английское слово, значение — перевод.

train(words) — Тренировка

Что делает:

- Перемешивает слова
- Задаёт по одному слову
- Показывает несколько вариантов перевода (правильный + случайные)
- Проверяет ответ и считает баллы

Пошагово:

1. Перемешивает список слов `random.shuffle(keys)`

2. Для каждого слова:

- Берёт правильный перевод
- Выбирает 3 случайных неправильных перевода
- Добавляет к ним правильный → перемешивает список

3. Показывает варианты и принимает номер

4. Проверяет ответ

5. В конце — показывает результат

```
def train(words):  
    if not words:  
        print("Словарь пуст. Добавьте слова сначала.")  
        return  
  
    keys = list(words.keys())  
    random.shuffle(keys)  
    score = 0  
  
    for eng in keys:  
        correct = words[____]  
        # создаем варианты ответов  
        all_answers = list(words.____())  
        all_answers.remove(correct)  
        options = random.sample(all_answers, min(3,  
len(____)))  
        options.append(correct)  
        random.shuffle(options)  
  
        print(f"\nЧто означает слово '{eng}'?")  
        for i, opt in enumerate(options):  
            print(f"{____ + 1}. {____}")
```

```
choice = input("Ваш выбор (номер): ")

if choice.isdigit() and 1 <= int(choice) <=
len(options):

    if options[_____ - 1] == correct:
        print("Верно!")
        score += 1
    else:
        print(f"Неверно. Правильный ответ:
{correct}")
    else:
        print("Неверный ввод.")

print(f"\nРезультат: {_____}/{_____} правильных
ответов.")
```